

A process can read data from the *read* end and the data can be written on the *write* end. In our example, the *who* command writes to the pipe, and the command *wc -l* reads from the pipe. This can be implemented by a C program, but I leave this for you to try, with some hints at the end of this section.

Here's the algorithm for a sender and receiver using a FIFO object.

Sender

Create a FIFO. (If already created then proceed).

Open the FIFO for Reading/Writing.

```
While (1) {
```

```
    Read from Key board.
```

```
    Write data read from user to FIFO.
```

```
    If User enters "exit"
then break the loop.
```

```
}
```

Delete FIFO and exit program.

Receiver

Create a FIFO. (If already created then proceed)

Open the FIFO for Reading/Writing

```
While (1) {
```

```
    Read data from FIFO
```

```
    Print the read data on the screen
```

```
    If received data is "exit" then break the loop.
```

```
}
```

Program terminates.

The C programs for the two blocks of pseudo-code above are in the downloaded archive as *fifo_sender.c* and *fifo_receiver.c*. Compile each, preferably naming the output as *sender* and *receiver*, run them in two different terminal windows and test them out.

Please note the function to create a FIFO is *mkfifo*. The name for the FIFO is accepted as a command-line argument to the program; the *S_IRWXU* flag is for specifying access permissions to the FIFO. You can check the man page (run *man 2 open* in a terminal) for the *open* system call, which will show you the detailed list of symbolic constants available for file access permissions. If you run an *ls*, you should see the created FIFO in the list.

Now, open the FIFO to get the file descriptor, which can be used for subsequent reading or writing to or from it. If you try to open a FIFO for reading, but no process has opened it for writing (or vice-versa), the program blocks in the call to open the FIFO. To avoid this, I have used the *O_RDWR* flag to specify opening in *read* or *write* mode. Other functions like *read/write*, which are used for files and even sockets, are known to you from our discussion of file systems. When the user enters *exit*, the sender breaks the loop, deletes the FIFO and exits.

Differences between FIFO and pipes

At the beginning of the section, I mentioned pipes, but we've just looked at an example of a FIFO. Is there any difference? Yes! Here, the communicating processes are not '*related*', but when using a pipe, you can only communicate between *related processes*—those with common ancestors or the result of a *fork* call. This is because a FIFO has a name (and so is sometimes called a *named pipe*) whereas a pipe does not have any name. You can try rewriting this code using a pipe. A hint: please read the man pages for the *pipe* and *dup* functions. You could also try writing the *who | wc -l*, or could try to enhance the simple shell, from the process part of this article, to support pipes.

System V and POSIX IPC objects

The IPC objects in Group 2 are called System V IPC objects, because they originate from System V UNIX. Most systems nowadays have POSIX implementations of these—and the functions used to create and use them in each implementation (System V and POSIX) differ. We will extend the earlier simple FIFO IPC example to use a System V message queue, and also to use System V shared memory along with a POSIX semaphore.

The question now is how to *uniquely identify* an IPC object that several processes use to communicate between each other? These processes may be unrelated and may run at different times. Here, System V takes a *key-based* approach (that is a number), while POSIX takes a *name-based* approach. This means that all processes that need to communicate through a System V IPC object must know a number that's used to index the particular IPC object. On the other hand, processes communicating via POSIX IPC objects need to know the object's name, which is like a filename (though the IPC object is not an ordinary file) to uniquely identify an IPC object they can use for communication.

The POSIX interface also allows for memory-based IPC objects (stored in memory). These name-based IPC objects can be used by *threads*, since